Learning PX4–GNSS Velocity Residuals from IMU Features: A Classical Machine Learning Benchmark

Meghna Nair, Pranjal Shukla, and Shreya Narayani
Department of Computer Science & Artificial Intelligence
Rishihood University, Sonipat, India

***Abstract*—Learning corrections from inertial measurements is attractive for unmanned aerial vehicles operating with intermittent satellite navigation. However, accurate prediction of a recorded estimator–reference residual is distinct from validated inertial drift compensation during a GNSS outage. This paper evaluates that distinction through an offline classical machine learning benchmark using 38,967 telemetry records from four PX4-associated logs. Three-dimensional velocity residuals are defined as the recorded PX4 velocity minus GPS reference velocity. A causal, gap-aware pipeline constructs 142 features from six inertial axes and two signal magnitudes. Ridge regression, random forest regression, and histogram gradient boosting are compared with zero-correction and training-mean baselines using recording-level partitions. On the 8,306-row evaluation log, zero correction achieves a mean of axis-wise root mean squared errors of 0.1472 m/s, compared with 0.1626 m/s for random forest, 0.1684 m/s for histogram boosting, and 0.1746 m/s for Ridge. The best learned correction therefore increases the aggregate error by 10.46%. A marked change in the vertical residual mean across recordings provides evidence of distribution shift, without identifying its physical cause. The study establishes a reproducible negative-result benchmark and specifies the provenance, validation, and reference requirements needed before extending the work to deep sequence models or genuine GNSS-denied evaluation.**

***Index Terms*—inertial measurement unit, velocity residual, UAV, regression, GNSS, temporal validation, negative result**

I. Introduction

Inertial measurement units (IMUs) provide high-rate measurements that support vehicle motion estimation when external positioning is intermittent. Accelerometers and gyroscopes are nevertheless affected by bias, noise, and calibration errors. Integration propagates these errors into velocity, attitude, and ultimately position. Learning-based inertial odometry has consequently investigated whether patterns in recorded motion can supplement physical propagation models [1]–[3].

For an unmanned aerial vehicle (UAV), a practical research question is whether a lightweight regressor can estimate a useful correction from measurements that remain available when global navigation satellite system (GNSS) measurements are unavailable. A supervised experiment requires both an input available at prediction time and a credible target. These requirements are not satisfied merely by removing GPS columns from a tabular dataset: the estimator output used to define the target may itself incorporate GPS aiding. Moreover, improving over another trained model does not establish that applying a correction is preferable to leaving the original estimate unchanged.

This study evaluates instantaneous PX4–GPS velocity discrepancies using recorded telemetry and a public implementation [4]. Its scope is offline classical machine learning. It neither implements a navigation stack nor integrates corrections into an extended Kalman filter (EKF). Deep learning remains future work.

The contribution is an auditable benchmark rather than a new navigation architecture. First, the dataset is checked for numerical integrity, label consistency, and temporal discontinuities. Second, a fixed set of causal IMU-derived features is evaluated with linear and tree-based regression methods under recording-level separation. Third, the comparison includes both a zero-correction baseline and a training-mean predictor, allowing the benefit of residual correction to be assessed directly. Finally, a negative finding is analyzed in relation to target shift, limited provenance, and validation design. This framing preserves the distinction between predicting recorded residuals and demonstrating operational GNSS-denied navigation improvement.

II. Related Work

A. Learning from Inertial Sequences

RIDI regresses velocity from histories of inertial measurements and uses the estimates to correct low-frequency acceleration bias before integration [1]. Its human-motion setting demonstrates the value of motion-dependent statistical structure, but does not establish that the same structure holds for aerial vehicles. RoNIN provides a neural inertial navigation benchmark with diverse human subjects and device configurations [5]. These studies motivate careful separation of recordings and operating conditions when evaluating generalization.

TLIO predicts three-dimensional displacement and uncertainty, incorporating the learned measurements into a tightly coupled filtering framework [2]. Its output is therefore more than a direct pointwise correction: uncertainty and temporal constraints participate in estimation. For autonomous drone racing, Cioffi *et al.* combine a temporal convolutional model with an inertial filter; their learned component uses mass-normalized collective thrust and gyroscope measurements [3]. Those additional dynamics-related inputs distinguish their formulation from the present CSV, which does not contain thrust or explicit attitude. Their reported navigation results cannot be transferred to this benchmark without reproducing the corresponding sensing and evaluation assumptions.

B. Classical Regression as a Benchmark

Ridge regression stabilizes least-squares estimation through an L_2 coefficient penalty [6]. This is relevant to engineered inertial features because adjacent lags and overlapping windows are strongly related by construction. Random forests aggregate randomized regression trees [7], while gradient boosting constructs an additive predictor through successive loss-reducing updates [8]. They provide complementary nonlinear alternatives without requiring a neural sequence architecture.

The present scikit-learn benchmark [9] evaluates established regressors against zero correction. Different references and evaluation settings preclude direct performance comparisons with published odometry systems. No novelty of the residual definition or general superiority over deep learning is claimed.

III. Problem Statement

Let $\mathbf{v}_t^{\mathrm{PX4}}, \mathbf{v}_t^{\mathrm{GPS}} \in \mathbb{R}^3$ denote the recorded estimator and GPS velocity vectors. Their components are interpreted as North, East, and Down (NED), subject to the provenance qualification in Section IV. The supervised target is

$$\mathbf{y}_t = \mathbf{v}_t^{\mathrm{PX4}} - \mathbf{v}_t^{\mathrm{GPS}} = \begin{bmatrix} \Delta v_{N,t} & \Delta v_{E,t} & \Delta v_{D,t} \end{bmatrix}^T. \quad (1)$$

A feature extractor ϕ summarizes the current and past inertial observations in the same continuous segment, producing $\mathbf{x}_t \in \mathbb{R}^{142}$. A fitted regressor f_θ predicts

$$\widehat{\mathbf{y}}_t = f_\theta(\mathbf{x}_t), \qquad \mathbf{v}_t^{\mathrm{corr}} = \mathbf{v}_t^{\mathrm{PX4}} - \widehat{\mathbf{y}}_t. \quad (2)$$

Positive target values indicate that the recorded estimator component exceeds the reference component. The target is neither an inter-sample velocity increment nor a direct accelerometer-bias measurement.

Equation (2) implies

$$\mathbf{v}_t^{\mathrm{corr}} - \mathbf{v}_t^{\mathrm{GPS}} = \mathbf{y}_t - \widehat{\mathbf{y}}_t. \quad (3)$$

Thus, for identical rows and axes, residual-prediction RMSE equals corrected-velocity RMSE relative to GPS. The offline correction calculation verifies the sign convention and its effect, but is not an independent downstream navigation trial. Position accuracy would require a validated trajectory reference and a defined integration or estimator procedure, neither of which is supplied by this dataset.

The importance of this distinction follows from the measurement model. In a simplified notation, specific force satisfies

$$\mathbf{f}_m^b = R_{nb}^T(\mathbf{a}^n - \mathbf{g}^n) + \mathbf{b}_a + \mathbf{n}_a, \quad (4)$$

where R_{nb} rotates the body frame into the navigation frame. A body-frame sensor value is not directly a gravity-free NED acceleration. The current pipeline neither estimates R_{nb} nor integrates Eq. (4). It learns a statistical mapping to the recorded residual, which may reflect estimator behavior, reference uncertainty, and alignment effects as well as inertial error.

TABLE I
Recording-level dataset audit and fixed partition

Log	Rows	Span (s)	Max gap (s)	Seg.	Role
51	10,277	70.400	2.315	11	Train
52	8,657	64.005	2.335	11	Val.
53	11,727	82.155	2.315	13	Train
54	8,306	57.560	2.315	9	Test

IV. Dataset Description

A. Schema and Provenance

The input CSV contains 38,967 rows and 31 columns: one recording identifier and 30 numeric fields. The inputs comprise three accelerometer axes, three gyroscope axes, and their two Euclidean magnitudes. Other fields contain timestamps, elapsed time, three GPS velocities, three PX4 velocities, GPS quality indicators, estimator validity flags, and the three precomputed targets. The file has no explicit attitude, position trajectory, thrust, or independently calibrated sensor-error label.

Units and frames follow the CSV names and documented PX4 conventions. PX4's `SensorCombined` message specifies microsecond timestamps, accelerometer channels in m/s^2, and angular-rate channels in rad/s in the forward–right–down body frame [10]. However, the repository does not include the original ULog files, extraction script, or complete fusion configuration. Consequently, these conventions are not independent proof of the exact CSV topic mapping or GPS aiding state. The four identifiers are treated as separate recording groups; independence of vehicles, sessions, and operating conditions is not established.

B. Numerical and Temporal Audit

There are no missing values, infinite numeric entries, or duplicate full rows. The target components reproduce Eq. (1) with a maximum absolute discrepancy of 2.23×10^{-16} m/s after rounding the observed maximum upward. This verifies arithmetic consistency, not the accuracy or synchronization of the source velocities. The `dead_reckoning` flag is zero throughout, while the listed velocity and position validity flags are one. These values do not by themselves identify all estimator aiding sources.

Table I summarizes the recordings. The median timestamp interval is 5 ms, corresponding to a nominal 200 Hz cadence. Multi-second gaps occur, with a maximum of 2.335 s. No continuous 200 Hz coverage is assumed. Applying the implemented gap threshold produces 44 segments. Repeated GPS values cannot be interpreted as independent high-rate reference measurements, and the native GPS update rate is not inferred from the merged row count.

Figure 1 shows the timing irregularity and target means. The mean Down residual changes from $+0.2754$ m/s in log 51 to -0.0443 m/s in log 54. The pooled training mean on that axis is $+0.1530$ m/s. These observations demonstrate target-distribution differences across recordings; they do not identify

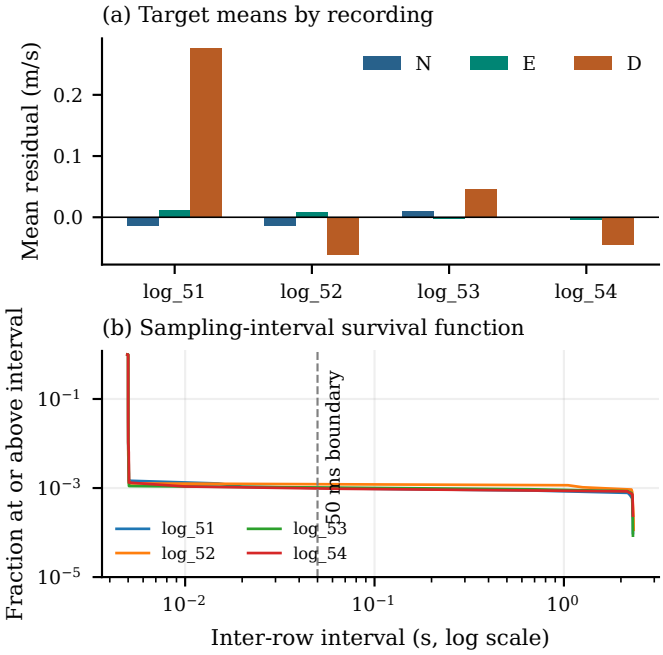


Fig. 1. Audit of the unchanged CSV. (a) Axis-wise target means vary across recordings. (b) Empirical survival curves show the fraction of intervals at least as large as the horizontal-axis value. Both axes in (b) are logarithmic; the dashed line marks the 50 ms segmentation threshold.

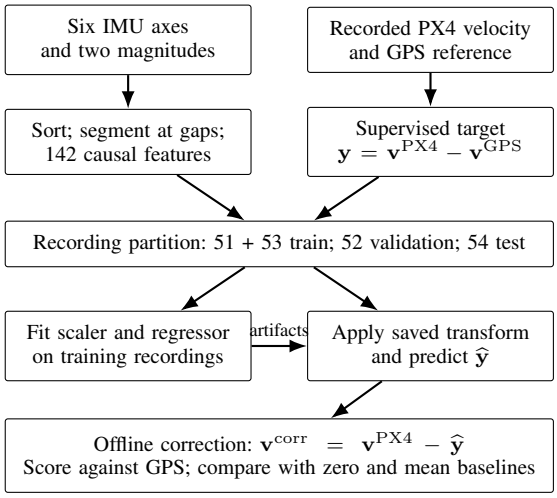


Fig. 2. Implemented offline benchmark. GPS and PX4 velocities provide labels and evaluation values; neither enters the 142-feature predictor. This separation does not establish that the recorded PX4 estimate was independent of GPS.

downwash, gravity compensation, GPS error, or any other particular cause.

V. Methodology

Figure 2 separates the predictor's permitted inputs from reference data used for supervision and scoring. All learned models receive the same feature columns and recording partitions. Existing PX4 velocities are used to construct labels and apply the offline correction, not as learned-model inputs.

A. Data Preprocessing

Rows are ordered by recording identifier and timestamp. The code excludes GPS velocities, speed, quality indicators, estimator velocities, and validity/status flags from the feature matrix. Recording identifiers and timestamps remain metadata. The three target columns are retained for supervision and explicitly excluded from predictors. Since the audited dataset is complete, no missing-value imputer or blanket outlier deletion is applied. Interquartile-range flags in the exploratory notebook serve as diagnostics, not automatic evidence that a manoeuvre sample is invalid.

A new continuous segment begins when an inter-row interval exceeds 50,000 μs. This 50 ms threshold is ten times the median interval and is a documented implementation choice rather than an optimized physical constant. All rolling statistics, lags, differences, and channel derivatives reset at segment boundaries. The procedure does not interpolate across gaps or manufacture additional reference measurements. All original rows are retained, enabling comparisons on identical evaluation rows.

For each model, standardization uses the training mean μ_j and standard deviation s_j for feature j:

$$z_{tj} = (x_{tj} - \mu_j)/s_j. \tag{5}$$

Validation and test rows use these same statistics. Scaling is important for the Ridge penalty and is retained for consistency in the tree scripts, although tree partitions generally do not require it. Within Ridge cross-validation, a new scaler is fitted in each training fold, preventing holdout statistics from entering the fitted transform.

B. Feature Engineering

The feature inventory is given in Table II. The eight base channels are six measured axes and two derived magnitudes, not eight independent sensor axes. Rolling windows contain 10, 50, or 100 samples. At nominal cadence they are conventionally described as approximately 50, 250, and 500 ms windows; actual elapsed support varies with retained intervals. Each window includes the current sample and available earlier samples in the same segment.

For a channel x, the rolling mean and range are

$$\mu_t^{(w)} = \frac{1}{n_t} \sum_{i \in \mathcal{W}_t} x_i, \quad r_t^{(w)} = \max_{i \in \mathcal{W}_t} x_i - \min_{i \in \mathcal{W}_t} x_i, \tag{6}$$

where $\mathcal{W}_t$ contains at most w samples and $n_t = |\mathcal{W}_t|$. Standard deviations use the implementation's sample-statistic convention; a single-sample window is assigned zero. Minima and maxima are included separately for the six axes.

First differences $x_t - x_{t-1}$ are sample differences, not derivatives per second. Separate acceleration-channel derivatives are

$$j_{k,t} = \frac{a_{k,t} - a_{k,t-1}}{(\tau_t - \tau_{t-1})10^{-6}}, \quad k \in \{x, y, z\}, \tag{7}$$

using timestamps τ interpreted in microseconds. At segment starts these features are set to zero. Equation (7) is a derivative

TABLE II
FEATURE INVENTORY USED BY EVERY LEARNED MODEL

Feature family	Count
Six axes and two Euclidean magnitudes	8
Mean/std.: 8 channels $\times$ 3 windows $\times$ 2	48
Min/max/range: $6 \times 3 \times 3$	54
First sample differences: 8 channels	8
Lags 1, 5, 10: 6 axes $\times$ 3	18
Time-normalized acceleration derivatives	3
Two XY angles and one norm ratio	3
Total predictor dimensions	142

of the recorded body-frame channel; it is not a verified navigation-frame trajectory jerk. In particular, it is not the second derivative of acceleration.

Lagged observations at 1, 5, and 10 samples supply short histories to static regressors. When a lag is unavailable at segment startup, the implementation substitutes the current channel value after forward filling available lag values. These causal startup substitutes are not historical observations. The remaining interactions are $\operatorname{atan2}(a_y, a_x)$, $\operatorname{atan2}(\omega_y, \omega_x)$, and $\|\mathbf{a}\|/(\|\boldsymbol{\omega}\| + 10^{-8})$. The angles are not vehicle attitude. The denominator offset prevents division by zero, not large ratios. No feature ablation is claimed.

VI. Proposed Classical ML Models

A. Reference Baselines

The zero predictor sets $\widehat{\mathbf{y}}_t = \mathbf{0}$, leaving PX4 velocity unchanged. The training-mean predictor sets $\widehat{\mathbf{y}}_t = \overline{\mathbf{y}}_{\text{train}}$ for every evaluation row. These distinguish a useful learned correction from both no intervention and a constant training-period offset. Neither predictor uses evaluation labels to estimate its output. In particular, the training-mean baseline must not be confused with the evaluation-set mean appearing in the definition of R^2.

B. Multi-output Ridge Regression

For standardized design matrix $Z \in \mathbb{R}^{n \times 142}$ and targets $Y \in \mathbb{R}^{n \times 3}$, Ridge solves

$$\min_{W,\mathbf{b}} \ \|Y - ZW - \mathbf{1}\mathbf{b}^T\|_F^2 + \alpha \|W\|_F^2. \tag{8}$$

This unnormalized objective matches the implemented penalty convention. Ridge provides a linear reference with regularization against unstable coefficients among correlated features [6]. There are 426 coefficients and three intercepts. The loss separates across output columns; multi-output support does not imply explicit estimation of cross-axis error covariance.

The candidate set is $\{0.01, 0.1, 1, 10, 50, 100, 500, 1000\}$. Leave-one-recording-out validation alternately holds out logs 51 and 53 while fitting on the other. The mean fold score selects $\alpha = 1000$, the upper endpoint of the tested grid. This is the best evaluated candidate, not evidence of a globally optimal regularization strength. The final estimator is fitted on both training logs.

C. Random Forest Regression

The forest predicts the average of B randomized regression trees,

$$\widehat{\mathbf{y}}(\mathbf{z}) = \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{z}). \tag{9}$$

Bootstrap sampling and random feature subsets diversify the trees [7]. The implementation uses $B = 150$, maximum depth 16, minimum split size 5, minimum leaf size 2, and `max_features=sqrt`. The random seed is 42. The estimator handles all three targets natively with shared partitions. Its impurity-based feature importance describes squared-error reduction in the fitted forest, not physical causality. Unequal target scales can influence the shared split criterion.

D. Histogram Gradient Boosting

For squared-error regression, boosting constructs

$$F_m(\mathbf{z}) = F_{m-1}(\mathbf{z}) + \eta h_m(\mathbf{z}), \tag{10}$$

where a new tree responds to the current model's loss gradient [8]. Histogram binning reduces the number of candidate split locations. The configured maximum iteration count is 120, learning rate 0.08, maximum depth 8, minimum leaf size 20, and leaf L_2 regularization 0.5, with seed 42. A `MultiOutputRegressor` wrapper fits three separate scalar models. Unspecified defaults include a maximum of 31 leaves and 255 nonmissing bins, with an additional bin reserved for missing values [11].

The forest and boosting settings are fixed in the training scripts; the repository does not document a comparable hyperparameter search for them. Accordingly, this experiment compares the implemented configurations rather than equally optimized algorithm families. No trained deep learning model is included, and the workflow diagram should not be interpreted as a neural architecture.

VII. Experimental Setup

A. Partition and Validation Protocol

Logs 51 and 53 provide 22,004 training rows, log 52 provides 8,657 validation rows, and log 54 provides 8,306 test rows. The same partitions and retained rows are used for every model and baseline. A recording identifier never occurs in multiple outer partitions. This reduces the overlap of highly correlated neighbouring measurements that would result from random row-wise splitting. It does not establish independence of recording conditions, and the evaluation log has already been inspected during development. It is therefore a held-out recording, not a pristine prospective test of all subsequent decisions.

Ridge selects its penalty within the training logs; log 52 is evaluated after fitting. Histogram boosting retains default early stopping with a 10% internal random validation subset because the training set exceeds 10,000 rows [11]. Log 52 is not used for stopping. This is a sequential-data validation limitation, although outer test labels remain excluded. All three estimators reached 120 iterations in the verification run.

B. Implementation and Reproducibility

The benchmark snapshot is commit `687dac1` of the public repository [4]. The main results are taken from its committed comparison and correction tables. The unchanged CSV has SHA-256 prefix `000e13ec4713b0ae`; its complete checksum and the source commit are preserved in the accompanying report package. Training scripts rebuild the deterministic features, fit scalers on training rows, serialize the estimators and scalers, and write parameter summaries and plots. The JSON configuration files record these runs; they are not input configuration files read by the training scripts.

An isolated verification run executed the EDA program, the preparation/training/comparison workflow, and 14 unit tests with Python 3.12.14, pandas 2.2.3, NumPy 2.3.5, and scikit-learn 1.8.0. The tests passed on freshly generated artifacts, and the EDA verified that the dataset checksum was unchanged. The principal ranking reproduced, although tree-model values differed slightly from the committed run. Both sets are reported separately below rather than silently replacing the repository results. Dependency lower bounds do not guarantee exact reproduction.

VIII. Evaluation Metrics

For axis k and prediction error $e_{ik} = \widehat{y}_{ik} - y_{ik}$, the metrics are

$$\mathrm{MAE}_k = \frac{1}{n}\sum_{i=1}^{n} |e_{ik}|, \quad \mathrm{RMSE}_k = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_{ik}^2}, \tag{11}$$

$$\mathrm{Bias}_k = \frac{1}{n}\sum_{i=1}^{n} e_{ik}, \qquad R_k^2 = 1 - \frac{\sum_i e_{ik}^2}{\sum_i (y_{ik} - \bar{y}_k)^2}. \tag{12}$$

Here $\bar{y}_k$ is the mean of the evaluated target values. A negative R_k^2 indicates worse squared error than that evaluation-set mean, which is not a deployable training-only baseline. The sign of corrected-velocity bias is opposite to prediction bias because of Eq. (3).

The primary aggregate is the mean of the three axis RMSEs:

$$\mathrm{RMSE}_{\mathrm{macro}} = \frac{1}{3}\sum_{k\in\{N,E,D\}} \mathrm{RMSE}_k. \tag{13}$$

It is distinct from pooled RMSE, $\sqrt{\sum_{i,k} e_{ik}^2/(3n)}$. The Ridge search uses pooled RMSE within each fold, whereas the final tables use Eq. (13); this distinction is retained in the experiment description. MAE and R^2 are likewise averaged across axes. Signed biases are inspected by axis because averaging signs can hide cancellation.

For a nonzero baseline error E_0, relative improvement is

$$I = 100\,(E_0 - E_{\mathrm{model}})/E_0. \tag{14}$$

Negative values indicate deterioration. Results are descriptive: no confidence interval or statistical-significance claim is derived by treating temporally adjacent rows as independent repetitions.

IX. Results and Discussion

A. Aggregate Benchmark

Table III and Fig. 3 summarize the committed evaluation on log 54. Zero correction obtains the lowest aggregate RMSE (0.1472 m/s) and MAE (0.1065 m/s). Random forest is the best learned model, with RMSE 0.1626 m/s and MAE 0.1189 m/s. Its RMSE is 10.46% higher than zero correction. Histogram boosting and Ridge increase aggregate RMSE by 14.40% and 18.58%, respectively. The constant training-mean correction also performs worse, with a 15.41% increase. Thus, selecting the best trained regressor would still select a harmful correction under the evaluated aggregate objective.

All reported mean R^2 values are negative. This is compatible with random forest outperforming the training-mean baseline: the reference for R^2 is the mean of the evaluation targets, not the mean learned from the training recordings. Even zero correction has slightly negative R^2 because the evaluation residual mean is not exactly zero. The distinction prevents the mistaken inference that a negative R^2 necessarily makes a model worse than every practical constant predictor.

B. Axis-wise Behavior and Distribution Shift

No learned model improves axis-wise RMSE over zero correction on log 54. Random forest increases North, East, and Down RMSE by approximately 1.50%, 2.06%, and 17.84%, respectively. Its North MAE is marginally lower than the zero baseline, but this isolated metric difference does not overturn its higher North RMSE or its worse aggregate behavior. The constant training-mean predictor lowers North RMSE by approximately 0.011%, while worsening the other axes. These small component-specific differences are reported without a significance claim.

The Down residual has substantially greater absolute error than the horizontal components. Its training mean is $+0.1530$ m/s, whereas its test mean is -0.0443 m/s. A constant prediction equal to the former therefore has signed prediction bias of approximately $+0.1973$ m/s on the test set. The committed random forest reduces this bias to $+0.1756$ m/s, but zero correction has only $+0.0443$ m/s of prediction bias on that axis. This arithmetic illustrates how a learned correction can preserve an offset associated with training conditions and worsen reference agreement on another recording.

The observed shift is a descriptive finding rather than a causal diagnosis. Changes in motion, estimator state, reference uncertainty, or synchronization could all contribute. There are no controlled measurements here that isolate gravity coupling, aerodynamic effects, or sensor temperature. Likewise, feature importance cannot identify which mechanism generated the residual. A causal explanation would require additional signals or a controlled experiment.

C. Correction Interpretation

The correction table forms $\mathbf{v}^{\mathrm{corr}} = \mathbf{v}^{\mathrm{PX4}} - \widehat{\mathbf{y}}$ on the same ordered rows and agrees with the residual metrics through Eq. (3). This verifies the correction sign, but is not an

TABLE III

COMMITTED RESULTS ON LOG 54. ERRORS ARE IN M/S; I IS IMPROVEMENT RELATIVE TO ZERO CORRECTION. RMSE IS THE ARITHMETIC MEAN OF AXIS RMSES, NOT POOLED RMSE. BOLD VALUES IDENTIFY THE BEST AGGREGATE ERRORS.

Method	Macro RMSE	Mean MAE	Mean R^2	RMSE_N	RMSE_E	RMSE_D	I (%)
Zero correction	**0.1472**	**0.1065**	−0.0127	0.1009	0.1022	0.2387	0.00
Random forest	0.1626	0.1189	−0.1709	0.1024	0.1043	0.2813	−10.46
Histogram boosting	0.1684	0.1252	−0.2552	0.1047	0.1091	0.2915	−14.40
Training mean	0.1699	0.1268	−0.2385	0.1009	0.1024	0.3065	−15.41
Ridge	0.1746	0.1319	−0.3869	0.1165	0.1153	0.2920	−18.58

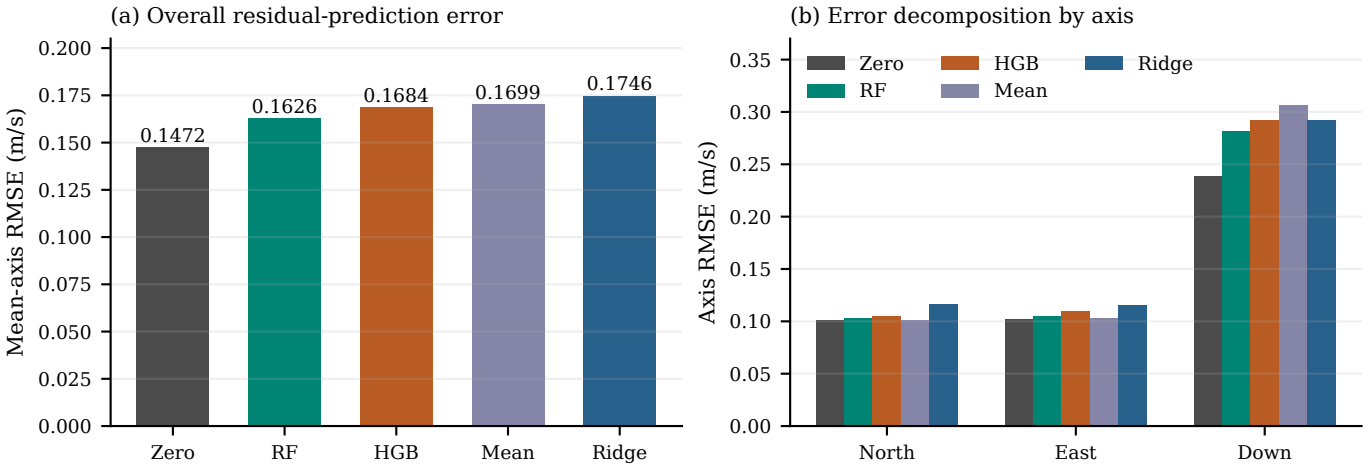


Fig. 3. Comparison using the committed predictions' metric tables. Zero correction is best overall. The Down axis contributes the largest absolute error; lower performance on that axis does not establish a particular physical cause. RF: random forest; HGB: histogram gradient boosting; Mean: training-mean predictor.

independent navigation experiment: no position trajectory is scored and no filter is replayed without GNSS updates.

The result is therefore narrower than the motivating navigation objective: the present IMU feature representation and configured regressors do not produce beneficial aggregate correction of the recorded PX4–GPS velocity disagreement on this evaluation log. The result does not prove that classical learning can never help inertial navigation, that all residuals are irreducible noise, or that a neural network will necessarily succeed.

D. Verification Run

Table IV distinguishes committed values from the isolated rerun. Ridge and the baselines agree to the displayed precision. Random forest and histogram boosting differ by approximately 0.00043 and 0.00227 m/s in aggregate RMSE. The saved metadata does not establish the cause. Both runs preserve the same ordering and negative conclusion; exact replication requires a locked environment and fuller artifact provenance.

TABLE IV

SEPARATE VERIFICATION RUN: MEAN-AXIS RMSE (M/S)

Method	Committed	Verification run
Zero correction	0.147245	0.147245
Random forest	0.162646	0.162213
Histogram boosting	0.168448	0.166180
Training mean	0.169940	0.169940
Ridge	0.174600	0.174600

X. LIMITATIONS AND THREATS TO VALIDITY

A. Reference Dependence and Observability

The principal limitation is target provenance. The recorded PX4 estimate may incorporate GPS or other aiding sources. Removing GPS from the learned input vector does not undo that dependence. The numerical target is a disagreement between an estimator output and an imperfect reference, not an independently observed inertial error. Unknown source-topic alignment and latency further limit its interpretation. A small temporal offset between velocity streams can create apparent residual structure during changing motion; arithmetic label consistency alone does not exclude this possibility.

Missing attitude and estimator history further limit observability. Similar body-frame readings can correspond to different navigation-frame components. Additional model capacity cannot by itself supply this missing information.

B. Dataset and Validation Constraints

Only four recording groups are available. Many adjacent rows are correlated, and the native GPS updates are not necessarily independent at the CSV cadence. The sample count

must not be treated as the number of independent experimental replications. The outer split prevents direct recording overlap, but its independence across vehicles and environmental conditions is unknown. Repeated inspection of log 54 also limits prospective claims. The absence of a fresh, independently collected evaluation session prevents a strong assessment of deployment generalization.

The random internal validation split in histogram boosting remains inconsistent with the outer grouping principle. In addition, tuning effort differs between algorithms, the Ridge tuning metric differs from the primary reported aggregation, and the largest Ridge penalty is selected at a grid boundary. These choices preclude claims of optimal or equal tuning. No feature ablation, nested multi-session validation, or uncertainty-calibrated correction experiment is available. The 50 ms gap threshold and the startup feature substitutes have also not been evaluated through sensitivity studies.

C. Computational and Statistical Claims

Batch prediction timings omit streaming feature computation and do not establish worst-case latency or memory use on a flight controller. They are not evidence of real-time readiness. Artifact tests support software integrity, not reference validity or out-of-distribution performance.

Independent-row bootstrap intervals would ignore temporal structure. Future uncertainty estimates should respect recording or contiguous-block dependence and require enough independent groups to be informative.

XI. Implications for Deep Learning and Future Work

A subsequent deep learning phase should test whether compact temporal convolutional or recurrent models extract transferable information beyond the fixed statistics. It should preserve the baselines, correction convention, and predeclared objective; sequence windows must respect gaps and partitions. No improvement is guaranteed by increasing capacity.

The priorities are to recover extraction provenance, verify source timing and frames, document aiding configuration, and acquire independent recordings. Raw-feature ablations should assess the value of expansion, and group-aware model selection should replace random internal validation. A fresh evaluation recording must remain untouched during development.

Genuine GNSS-denied evaluation requires a known outage in the estimator's data path and an independent reference retained for scoring. Hiding GPS columns from a predictor does not reproduce that condition. Trajectory-level metrics are needed for position claims, and uncertainty-aware estimator integration requires separate validation. These extensions are not implemented results of this benchmark.

XII. Conclusion

This paper evaluated classical regression for predicting PX4–GPS velocity residuals from causal IMU-derived features. The benchmark contains 38,967 rows, 142 predictor dimensions, three learned models, and two essential baselines, with outer partitions defined by recording. Gap-aware preprocessing and explicit target verification support reproducible computation, while the missing raw-log provenance limits physical interpretation. On log 54, zero correction achieves mean-axis RMSE of 0.1472 m/s, outperforming random forest, histogram boosting, and Ridge. The best learned correction increases aggregate error by 10.46%. An isolated rerun preserves this conclusion, although tree-model scores are not numerically identical. The defensible outcome is a negative residual-prediction benchmark, not validated GNSS-denied drift reduction. It provides both a baseline for subsequent sequence-learning experiments and concrete requirements for stronger data, reference, and validation design.

References

[1] H. Yan, Q. Shan, and Y. Furukawa, "RIDI: Robust IMU double integration," in *Proc. European Conf. Computer Vision (ECCV)*, 2018, pp. 621–636. [Online]. Available: https://openaccess.thecvf.com/content_ECCV_2018/html/Hang_Yan_RIDI_Robust_IMU_ECCV_2018_paper.html

[2] W. Liu, D. Caruso, E. Ilg, J. Dong, A. I. Mourikis, K. Daniilidis, V. Kumar, and J. Engel, "TLIO: Tight learned inertial odometry," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.

[3] G. Cioffi, L. Bauersfeld, E. Kaufmann, and D. Scaramuzza, "Learned inertial odometry for autonomous drone racing," *IEEE Robotics and Automation Letters*, vol. 8, no. 5, pp. 2684–2691, 2023.

[4] M. Nair, P. Shukla, and S. Narayani, "ML-assisted IMU drift reduction for GNSS-denied UAV navigation," GitHub repository, 2026, snapshot: commit 687dac12be7a5e44d7453522400a514573319c5e. [Online]. Available: https://github.com/heymegzz/vio-drift-reduction

[5] S. Herath, H. Yan, and Y. Furukawa, "RoNIN: Robust neural inertial navigation in the wild: Benchmark, evaluations, and new methods," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2020, pp. 3146–3152.

[6] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] PX4 Development Team, "SensorCombined (uORB message)," PX4 User Guide, accessed: Nov. 2, 2026. [Online]. Available: https://docs.px4.io/main/en/msg_docs/SensorCombined

[11] scikit-learn developers, "HistGradientBoostingRegressor," scikit-learn 1.8.0 documentation, accessed: Nov. 2, 2026. [Online]. Available: https://scikit-learn.org/1.8/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html